AstraQuantum Tech

Web Application Security

OWASP Top 10 Foundations - Beginner Track · PortSwigger Web Security Academy Labs

Name	Muhammad Raza
Company	AstraQuantum Tech
Program	Offensive Security Training - Beginner Track
Report	Week 4 - Web Application Security (OWASP Top 10 Foundations)
Date	September 5, 2026

1. Introduction & Objectives

This report documents the completion of the AstraQuantum Tech Week 4 assignment, Task WEBSEC-W01-BEGINNER: Web Application Security. The objective of this task was to build a working foundation in the OWASP Top 10 by completing a set of Apprentice-level labs on the PortSwigger Web Security Academy. Each lab was approached end-to-end - recognising the vulnerability, exploiting it manually using Burp Suite, and reasoning through its real-world impact beyond the immediate lab scenario.

Four labs were completed in total: two examples of Injection (A03), and two examples of Broken Access Control (A01). The fourth lab was substituted from the originally assigned CSRF exercise, as explained in that lab's section below, so that the full set could be completed using Burp Suite Community Edition without loss of learning value.

2. Tools Used

- Browser (Chromium-based, e.g. Microsoft Edge) - used to interact with each PortSwigger Web Security Academy lab instance.
- Burp Suite Community Edition - proxied all browser traffic through 127.0.0.1:8080 to inspect and modify requests in Proxy/Repeater.
- PortSwigger Web Security Academy - free, browser-based lab environment; no local VM or Docker required.

3. Lab Walkthroughs

Lab 1 - SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

OWASP CATEGORY **A03 - Injection**

PAYLOAD `' OR 1=1--`

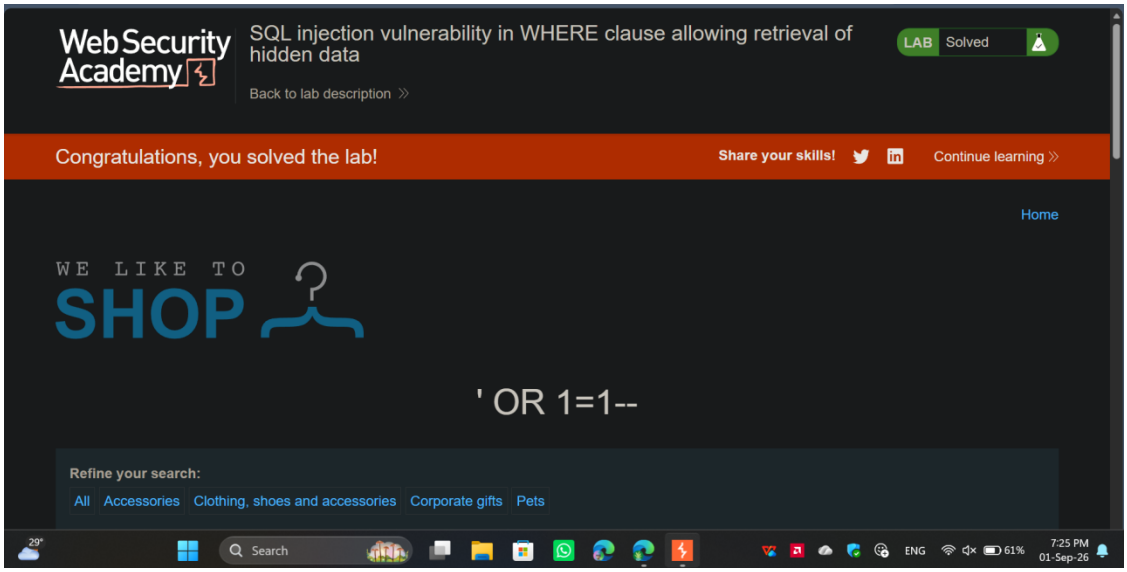

Figure 1.1 - Lab solved: SQL injection payload ' OR 1=1-- returns all hidden products.

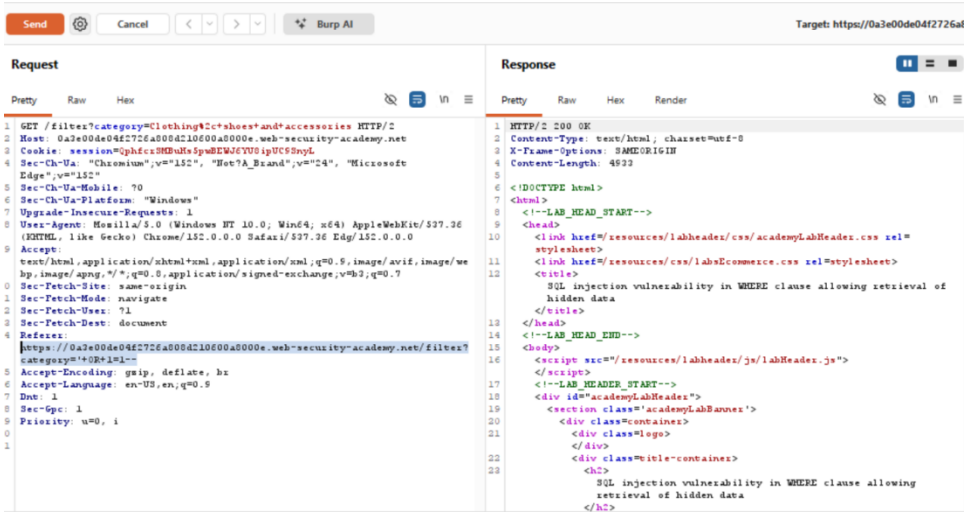

Figure 1.2 - Burp Suite Repeater showing the payload injected into the category parameter via the Referer header.

Explanation

This vulnerability exists because the web application concatenates user-supplied input from the category parameter directly into the backend SQL query without sanitization or parameterization. By injecting ' OR 1=1--, the attacker terminates the intended string and forces a universally true condition, while the -- comments out the remainder of the original query. Beyond this specific lab scenario, an attacker could leverage this to bypass authentication mechanisms, expose sensitive hidden records, or systematically map the entire database structure.

Mitigation

Implement parameterized queries (prepared statements) on the backend to ensure all user input is handled strictly as string data rather than executable SQL code.

Lab 2 - Cross-Site Scripting (XSS): Reflected XSS into HTML Context with Nothing Encoded

OWASP CATEGORY A03 — Injection

```
PAYLOAD <script>alert(1)</script>
```

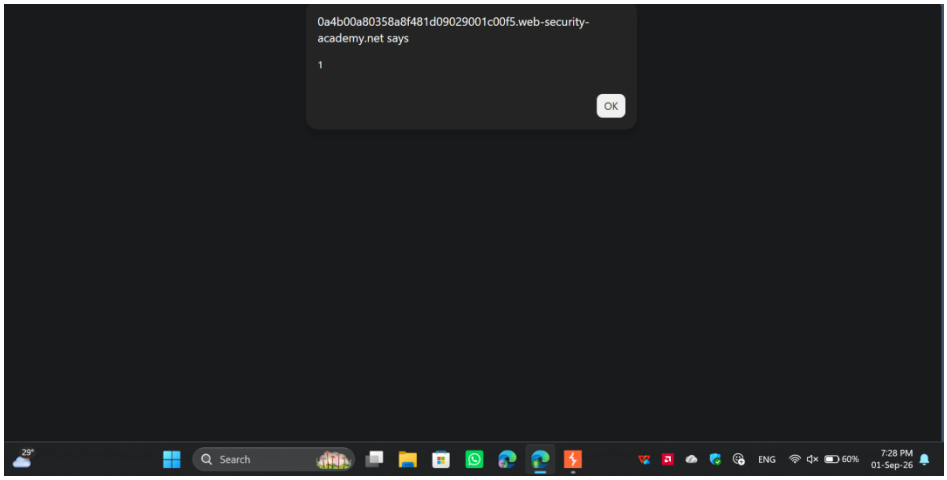

Figure 2.1 - JavaScript alert box firing, confirming the injected <script> executed in the browser.

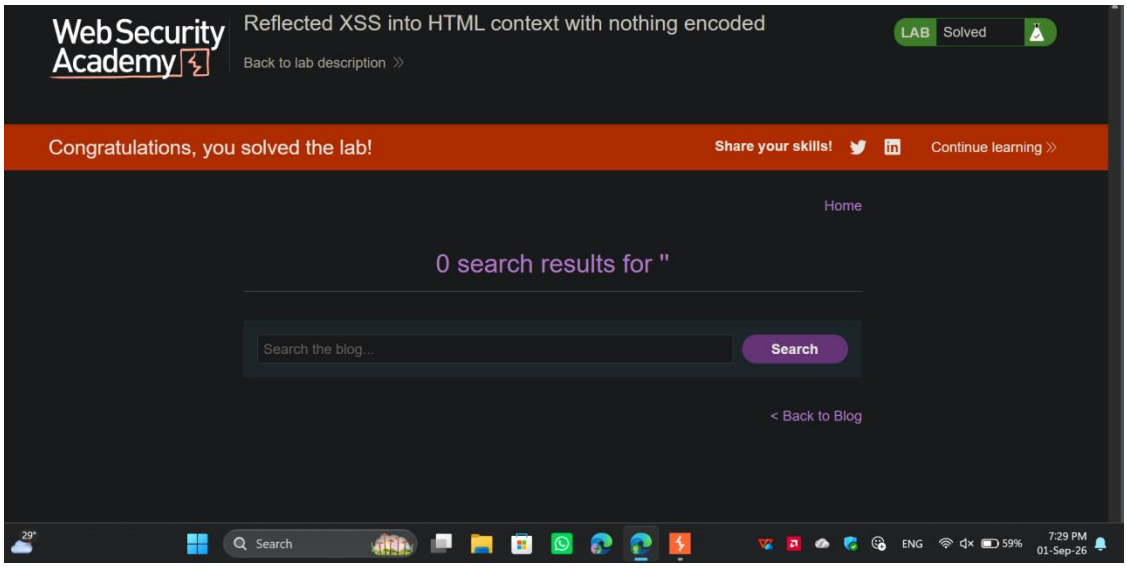

Figure 2.2 - Lab solved: reflected XSS confirmed via the search results page.

Explanation

This vulnerability occurs because the application takes user input from the search parameter and echoes it directly into the HTML response without any sanitization or encoding. Because the input is not treated as raw text, the victim's browser interprets the injected <script> tags as executable code. In a real-world scenario, an attacker could craft a malicious link containing this payload and send it to a victim; when clicked, the script executes within the victim's session, potentially allowing the attacker to steal session cookies, capture keystrokes, or perform unauthorized actions on the user's behalf.

Mitigation

Implement strict context-aware output encoding on the server side before reflecting any user-supplied data into the browser. For example, converting the angle-bracket characters into their HTML entity equivalents ensures the browser renders the data safely as text rather than executing it as code.

Lab 3 - Unprotected Admin Functionality

OWASP CATEGORY A01 - Broken Access Control

PAYLOAD	`Navigating directly to the hidden path /administrator-panel`

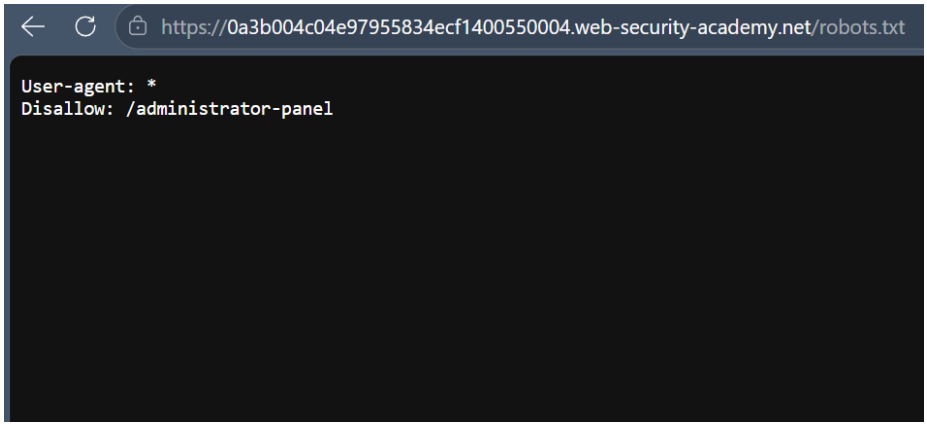

Figure 3.1 - robots.txt discloses the hidden /administrator-panel path.

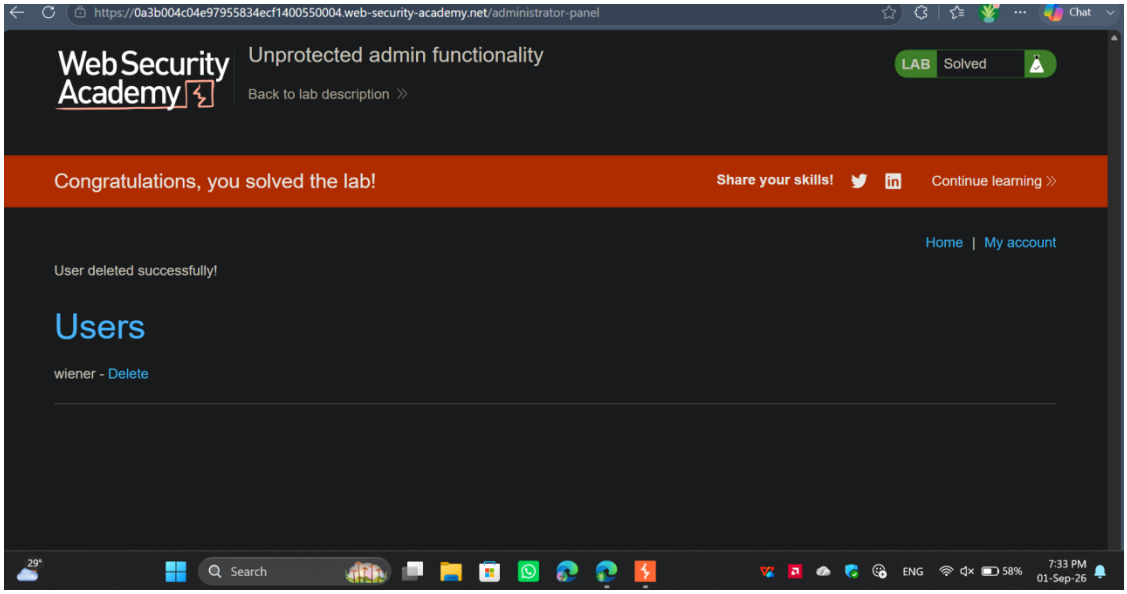

Figure 3.2 - Lab solved: admin panel reached directly, user account deleted with no authorization check.

Explanation

This vulnerability exists because the application relies entirely on "security by obscurity" by simply hiding the admin panel URL, rather than enforcing actual server-side authorization checks. By reviewing the publicly accessible robots.txt file, an attacker can easily discover the hidden directory. In a real-world scenario, anyone who finds this URL can access high-privileged administrative functions, allowing them to delete user accounts, modify configurations, or completely take over the application.

Mitigation

Implement strict server-side role-based access control (RBAC) on every administrative endpoint, ensuring that every request explicitly validates the user's session and verifies their administrative privileges before executing the requested action.

Lab 4 - File Path Traversal, Simple Case (Optional Alternative)

Reason for Substitution

The originally assigned CSRF vulnerability lab required the automated "Generate CSRF PoC" tool, which is exclusive to Burp Suite Professional. Following instructor guidance, this optional bonus lab was substituted so it could be completed using Burp Suite Community Edition.

OWASP CATEGORY **A01 - Broken Access Control**

PAYLOAD `../../../etc/passwd`

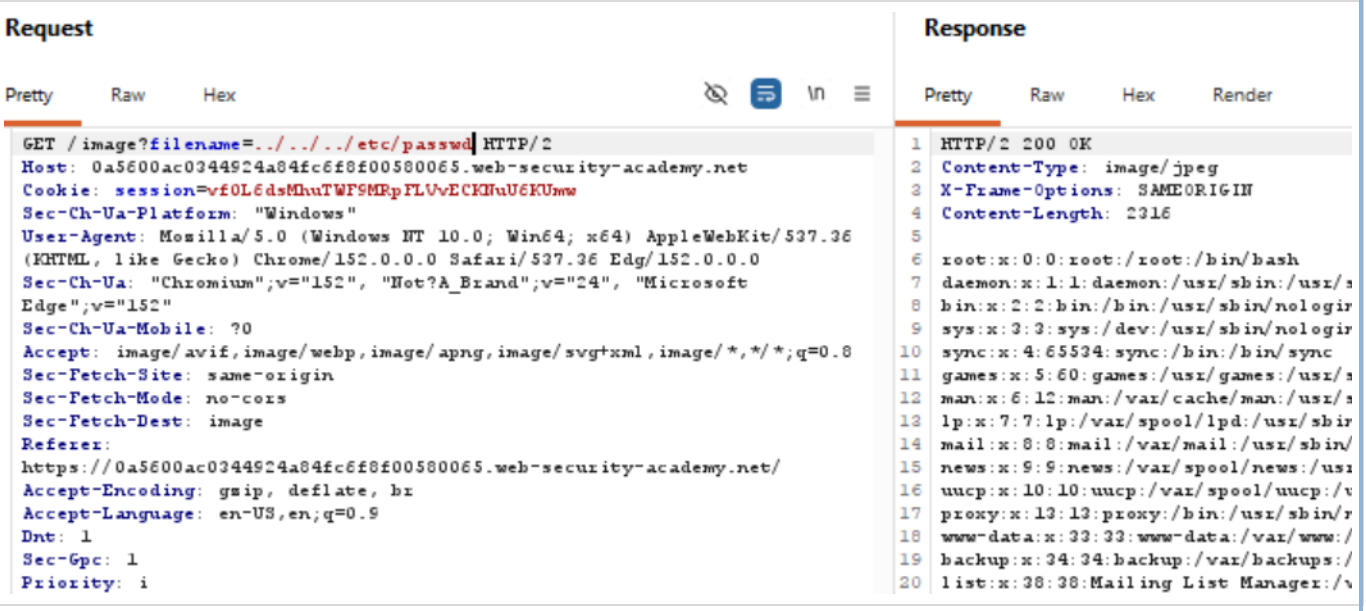

Figure 4.1 - Burp Suite request/response: filename=../../../etc/passwd returns the server's /etc/passwd file.

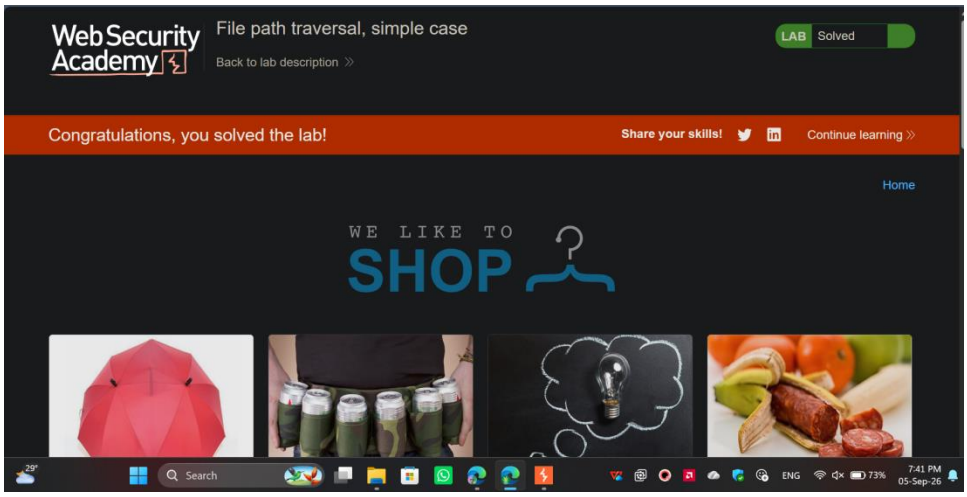

Figure 4.2 - Lab solved: file path traversal confirmed.

Explanation

The application processes the filename parameter to fetch image files but fails to strip or sanitize directory traversal sequences (../). By injecting these sequences, an attacker forces the backend server to navigate up the directory tree and retrieve arbitrary files from the file system. In a real-world scenario, this flaw allows attackers to steal highly sensitive operating system files, application source code, or configuration files containing hardcoded database credentials.

Mitigation

Validate all user-supplied filenames against a strict allowlist of permitted files. Additionally, resolve the absolute path of the requested file using built-in file system APIs and verify that it resides strictly within the intended base directory before granting read access.

4. Summary Table

The table below maps each completed lab to its corresponding OWASP Top 10 category.

#	Lab	OWASP Top 10 Category
1	SQL injection vulnerability in WHERE clause allowing retrieval of hidden data	A03 - Injection
2	Cross-site scripting (XSS): reflected XSS into HTML context with nothing encoded	A03 - Injection
3	Unprotected admin functionality	A01 - Broken Access Control
4	File path traversal, simple case (optional - substituted for CSRF)	A01 - Broken Access Control

5. Conclusion & Lessons Learned

Completing these four labs built a practical, end-to-end foundation across two of the most prevalent OWASP Top 10 categories. The Injection labs (SQL injection and reflected XSS) reinforced that the root cause in both cases is the same pattern - untrusted input reaching a sensitive sink (a SQL query or an HTML response) without being treated strictly as data. The Broken Access Control labs (the unprotected admin panel and the file path traversal substitution) reinforced that hiding a resource is not the same as protecting it - real security requires the server to enforce authorization and validate input on every request, not rely on obscurity.

Working through each vulnerability manually in Burp Suite - capturing the request, modifying it in Repeater, and observing the raw response - made the mechanics of each attack far clearer than reading about them in isolation. It also made the corresponding mitigations easier to internalise: parameterized queries, context-aware output encoding, server-side RBAC checks, and strict filename allowlisting are not abstract advice but direct, traceable fixes for the exact flaws exploited in each lab.

The main takeaway going into future weeks is that most of these vulnerabilities share a common root: user input is trusted somewhere it shouldn't be. Carrying that lens into the next set of labs - checking, for every input, where it ends up and whether it is validated before it gets there - will be the primary focus going forward.